Université de Nouakchott
Faculté des Sciences et Techniques (FST)
Master 1 — NLP / Computer Vision
Module : Traitement Automatique du Langage Naturel (NLP)
Année universitaire 2025–2026



Conception d'un assistant conversationnel RAG

pour le service de monnaie électronique **Bankily**

Système question-réponse hybride (FAISS + BM25 + re-ranking)
appuyé sur le modèle de langage Mistral-7B

Étudiant : Mohamed Mahmoud Said
Matricule : C34666
Encadrant : M. Mohamed El Ghaly
Domaine choisi : Documentation officielle et services de Bankily (monnaie électronique en Mauritanie)
Dépôt GitHub : `https://github.com/MdMahmoudSaid/rag-bankily-assistant`

15 juin 2026

Table des matières

1 Introduction **3**
1.1 Contexte et motivation . . . 3
1.2 Objectifs . . . 3
1.3 Périmètre du rapport . . . 3

2 État d'avancement **3**
2.1 Ce qui fonctionne . . . 3
2.2 Pistes d'amélioration . . . 4

3 Architecture du système **4**
3.1 Phase d'indexation (hors ligne) . . . 4
3.2 Phase de réponse (en ligne) . . . 4

4 Base de connaissances et préparation **5**
4.1 Source et contenu . . . 5
4.2 Nettoyage . . . 5
4.3 Découpage en fragments . . . 6
4.4 Exemples de documents . . . 6

5 Implémentation **6**
5.1 Choix techniques . . . 6
5.2 Pipeline de récupération et de génération . . . 7
5.3 Interface utilisateur (Gradio) . . . 7
5.4 Fonctions principales . . . 7

6 Méthodologie d'évaluation **8**
6.1 Jeu de questions . . . 8
6.2 Métriques de récupération . . . 8
6.3 Métriques de génération . . . 8
6.4 Reproductibilité . . . 8

7 Résultats et visualisations **8**
7.1 Performance de la récupération . . . 8
7.2 Qualité et temps de génération . . . 9

8 Apport du RAG : comparaison avant / après **11**
8.1 Question 1 : « C'est quoi Bankily ? » . . . 11
8.2 Question 2 : « Comment ouvrir / valider un compte Bankily ? » . . . 11
8.3 Synthèse de la comparaison . . . 12

9 Discussion **12**
9.1 Points forts . . . 12
9.2 Limites et travail en cours . . . 12
9.3 Alignement avec le sujet . . . 12

10 Conclusion et perspectives **13**
10.1 Perspectives . . . 13

11 Références **13**

A Extraits de code **13**
A.1 Prompt structuré (génération) 13
A.2 Recherche hybride (FAISS + BM25) 14
A.3 Re-ranking par cross-encoder 14
A.4 Métriques de récupération 14

B Exemples question-réponse **14**
B.1 Exemple 1 — définition d'un service 14
B.2 Exemple 2 — question procédurale 14
B.3 Exemple 3 — sources multiples 15

1 Introduction

1.1 Contexte et motivation

Ce travail s'inscrit dans le cadre du module de Traitement Automatique du Langage Naturel du Master 1 NLP/CV. Il a pour but de concevoir et d'évaluer un assistant capable de répondre, en langage naturel, aux questions des usagers du service de monnaie électronique **Bankily**. Le système repose sur l'approche *Retrieval-Augmented Generation* (RAG), qui combine une base documentaire interrogeable et un grand modèle de langage (LLM) afin de produire des réponses ancrées dans des sources vérifiables plutôt que générées de mémoire par le modèle.

Le choix du domaine Bankily est motivé par un besoin concret en Mauritanie : l'inclusion financière s'appuie largement sur la monnaie mobile, mais l'information utile (ouverture de compte, tarifs, transferts, paiements de factures, services comme El Lewha) est dispersée et parfois difficile d'accès pour l'utilisateur. Un assistant qui restitue cette information de manière fiable, dans la langue de la question, constitue donc un cas d'usage à la fois pédagogique et utile.

1.2 Objectifs

Le projet vise à couvrir la chaîne complète d'un système RAG :

- constitution d'une base de connaissances à partir du site officiel de Bankily ;
- nettoyage, segmentation en fragments et double indexation (vectorielle et lexicale) ;
- mise en place d'une récupération hybride suivie d'un re-ranking de précision ;
- génération de réponses contextuelles avec gestion de l'historique de conversation ;
- déploiement d'une interface conversationnelle ;
- évaluation quantitative séparée du module de récupération et du module de génération.

1.3 Périmètre du rapport

Ce document décrit l'état du projet au moment de la remise. La base de connaissances, les index, le pipeline hybride, le module de génération et les deux campagnes d'évaluation (récupération et génération) sont opérationnels et produisent des résultats mesurables. L'interface Gradio a été développée et testée. Le rapport est organisé comme suit : architecture du système (section 3), base de connaissances (section 4), implémentation et choix techniques (section 5), protocole d'évaluation (section 6), résultats et graphiques (section 7), puis discussion et conclusion (sections 9 et 10).

2 État d'avancement

Le tableau 1 distingue les composants terminés et les pistes restant à explorer.

2.1 Ce qui fonctionne

- Chargement de la base Bankily (fichiers `.md` structurés par titre et contenu), découpage en fragments de 800 caractères avec 100 caractères de recouvrement.
- Double indexation : FAISS pour la similarité sémantique (embeddings BGE-M3 normalisés) et BM25 pour la correspondance lexicale.
- Récupération hybride (fusion des deux listes, $k = 10$) puis re-ranking par cross-encoder pour ne conserver que les trois fragments les plus pertinents.
- Génération de réponses par Mistral-7B-Instruct quantifié en 4-bit, avec consigne de répondre dans la langue de la question et de ne pas inventer de réponse.
- Conservation d'un historique de conversation (quatre derniers échanges) pour les questions de suivi.

Table 1: Suivi des phases du projet (juin 2026).

Composant	Statut	Livrable principal
Collecte de la base de connaissances	Terminé	dossier `data/` (~35 fichiers `.md`)
Nettoyage et découpage en fragments	Terminé	`load_docs` + splitter (800/100)
Index vectoriel FAISS	Terminé	`vector_db` (BGE-M3)
Index lexical BM25	Terminé	`bm25` (rank_bm25)
Récupération hybride + re-ranking	Terminé	`hybrid_search`, `get_reranked_documents`
Pipeline RAG avec mémoire	Terminé	`ask_bankily_with_memory`
Interface conversationnelle Gradio	Terminé	application `gr.Blocks`
Jeu d'évaluation (20 questions)	Terminé	`eval_dataset`
Évaluation récupération (P@3, R@3)	Terminé	`df_retrieval`
Évaluation génération (BERT-F1, ROUGE-L, temps)	Terminé	`df_gen`

- Évaluation automatique séparée : Precision@3 / Recall@3 pour la récupération ; BERTScore-F1, ROUGE-L et temps de réponse pour la génération.

2.2 Pistes d'amélioration

- Persistance de l'index FAISS sur disque pour éviter la reconstruction à chaque session.
- Optimisation de la latence de génération (le LLM 7B quantifié reste le goulot d'étranglement).
- Enrichissement de la base de connaissances et harmonisation des intitulés de fichiers pour fiabiliser l'attribution des sources « gold ».

3 Architecture du système

Le système suit une architecture RAG en deux temps : une **phase d'indexation hors ligne**, exécutée une fois (ou à chaque mise à jour de la base), et une **phase de réponse en ligne**, déclenchée à chaque question. La particularité du système est une récupération *hybride* (sémantique + lexicale) suivie d'un *re-ranking*, illustrée en figure 1.

3.1 Phase d'indexation (hors ligne)

1. **Base de connaissances** : fichiers Markdown extraits du site officiel de Bankily, chacun comportant un titre et un contenu.
2. **Nettoyage et découpage** : extraction du titre/contenu puis segmentation par `RecursiveCharacterTextSp` en fragments de 800 caractères avec un recouvrement de 100 caractères.
3. **Embeddings** : chaque fragment est vectorisé avec `BAAI/bge-m3` (vecteurs normalisés), exécuté sur GPU.
4. **Double index** : les vecteurs alimentent un index FAISS ; en parallèle, un index BM25 est construit sur le corpus tokenisé pour la recherche par mots-clés.

3.2 Phase de réponse (en ligne)

1. La question est traitée par la recherche hybride : FAISS renvoie les fragments sémantiquement proches, BM25 les fragments lexicalement proches, et les deux listes sont fusionnées (doublons supprimés, $k = 10$).
2. Un cross-encoder (`ms-marco-MiniLM-L-6-v2`) re-évalue chaque paire (question, fragment) et ne conserve que les trois fragments les mieux classés.

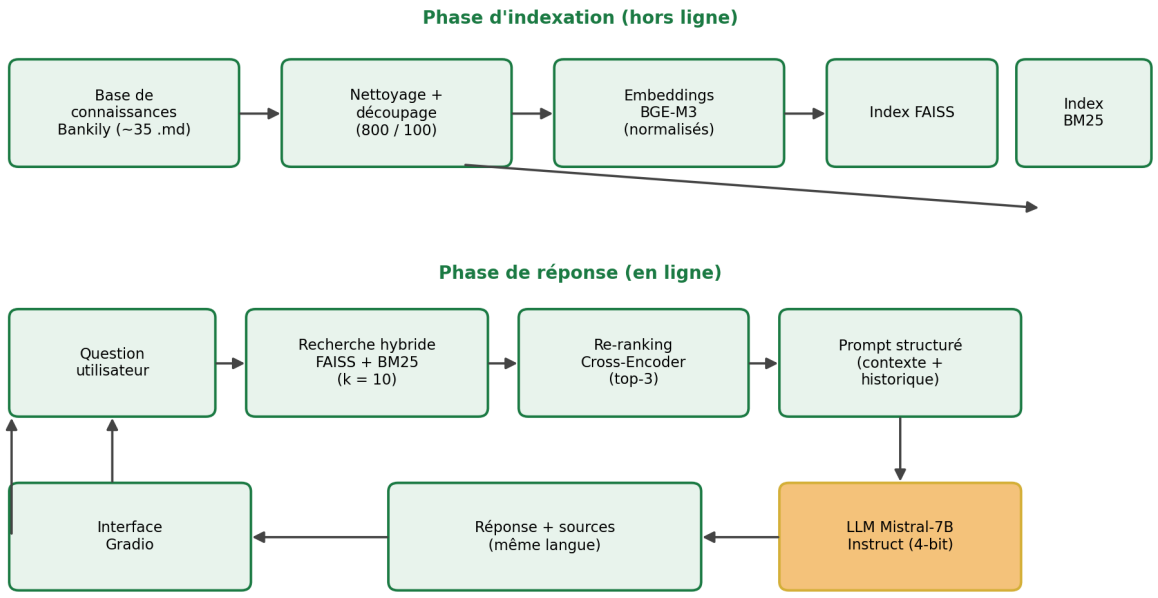


Figure 1: Architecture du pipeline RAG Bankily : indexation hors ligne (FAISS + BM25) et réponse en ligne (recherche hybride, re-ranking, génération Mistral-7B).

3. Ces fragments, l'historique récent et la question sont assemblés dans un prompt structuré.
4. Le prompt est soumis à Mistral-7B-Instruct (4-bit) qui génère la réponse dans la langue de la question.
5. La réponse et les sources sont restituées dans l'interface Gradio.

4 Base de connaissances et préparation

4.1 Source et contenu

La base de connaissances est constituée à partir du site officiel de Bankily et de sa documentation publique (FAQ, grilles tarifaires, descriptions de services). Elle couvre les principaux usages : ouverture de compte, tarification, transferts locaux et internationaux, retraits, paiement de factures et de la vignette, recharge téléphonique, paiement marchand, ainsi que les services spécifiques tels qu'El Lewha (tontine digitale) ou la présentation de la Banque Populaire de Mauritanie (BPM). Le dossier contient environ **35 fichiers** Markdown, ce qui dépasse la contrainte minimale de 30 documents fixée par le sujet.

4.2 Nettoyage

Chaque fichier est lu puis structuré en un objet `Document` : le titre est extrait via une expression régulière (motif `Title:`) et le corps via le séparateur `Content:`. Les métadonnées conservées (titre, nom de fichier source) permettent ensuite d'afficher et d'évaluer la provenance des réponses.

4.3 Découpage en fragments

Les documents sont segmentés par `RecursiveCharacterTextSplitter` en fragments de 512 à 800 caractères ; la configuration retenue est de **800 caractères avec 100 caractères de recouvrement**. Ce recouvrement réduit le risque qu'une information utile soit coupée à la frontière de deux fragments. La taille volontairement plus grande que la moyenne s'explique par la nature des réponses Bankily, souvent procédurales (étapes à suivre), qui gagnent à rester dans un même fragment.

4.4 Exemples de documents

Le tableau 2 liste un extrait des fichiers de la base, identifiés par leur nom.

Table 2: Extrait des fichiers de la base de connaissances Bankily.

Fichier	Fichier
003_faq_general	004_account_opening
006_pricing_account_and_transfers	007_pricing_withdrawals
009_pricing_free_services	010_local_money_transfer
013_remitly	014_ria
015_merchant_payment	016_bill_payment_utilities
017_phone_recharge	018_el_lewha_tontine
019_request_money	020_gimtel_recharge
021_vignette_payment	023_minors_account
026_bpm_presentation_generale	...

5 Implémentation

5.1 Choix techniques

Le tableau 3 résume les briques techniques et leur justification.

Table 3: Choix techniques et justifications.

Composant	Choix	Justification
Langage	Python (Google Colab, GPU)	écosystème NLP mature, GPU disponible pour le 7B.
Embeddings	`BAAI/bge-m3`	modèle multilingue performant, adapté au français et à l'arabe.
Récupération	FAISS + BM25 (hybride)	combine sémantique et mots-clés pour limiter les manques.
Re-ranking	cross-encoder `ms-marco-MiniLM-L-6-v2`	affine la pertinence des fragments retenus.
LLM	`Mistral-7B-Instruct-v0.2` (4-bit nf4)	open-weight, quantifié pour tenir en mémoire GPU.
Orchestration	LangChain	assemblage documents / prompt / LLM.
Interface	Gradio	déploiement rapide d'un chat web.
Découpage	800 / overlap 100	compromis granularité / réponses procédurales.
Top-k	10 récupérés, 3 conservés	large rappel puis filtrage par re-ranking.

5.2 Pipeline de récupération et de génération

Trois fonctions structurent la logique métier :

- `hybrid_search()` : interroge FAISS et BM25, fusionne et déduplique les résultats ;
- `get_reranked_documents()` : score chaque paire (question, fragment) avec le cross-encoder et conserve le top-3 ;
- `ask_bankily_with_memory()` : assemble contexte + historique + question dans le prompt, appelle Mistral-7B et met à jour l'historique.

Le prompt impose deux contraintes au modèle : répondre uniquement à partir des documents fournis (et déclarer son ignorance le cas échéant) et répondre dans la langue de la question (français, arabe, etc.), ce qui réduit les hallucinations. Un extrait figure en annexe A.1.

5.3 Interface utilisateur (Gradio)

L'assistant est exposé via une interface Gradio aux couleurs de Bankily (vert et or), affichant le logo de la marque, un avatar utilisateur et une zone de conversation. Le module RAG complet (`ask_bankily_with_memory`) y est branché, l'historique permettant les questions de suivi. Le périphérique de calcul (CUDA/CPU) est indiqué à l'utilisateur au démarrage. La figure 2 illustre une session de démonstration.

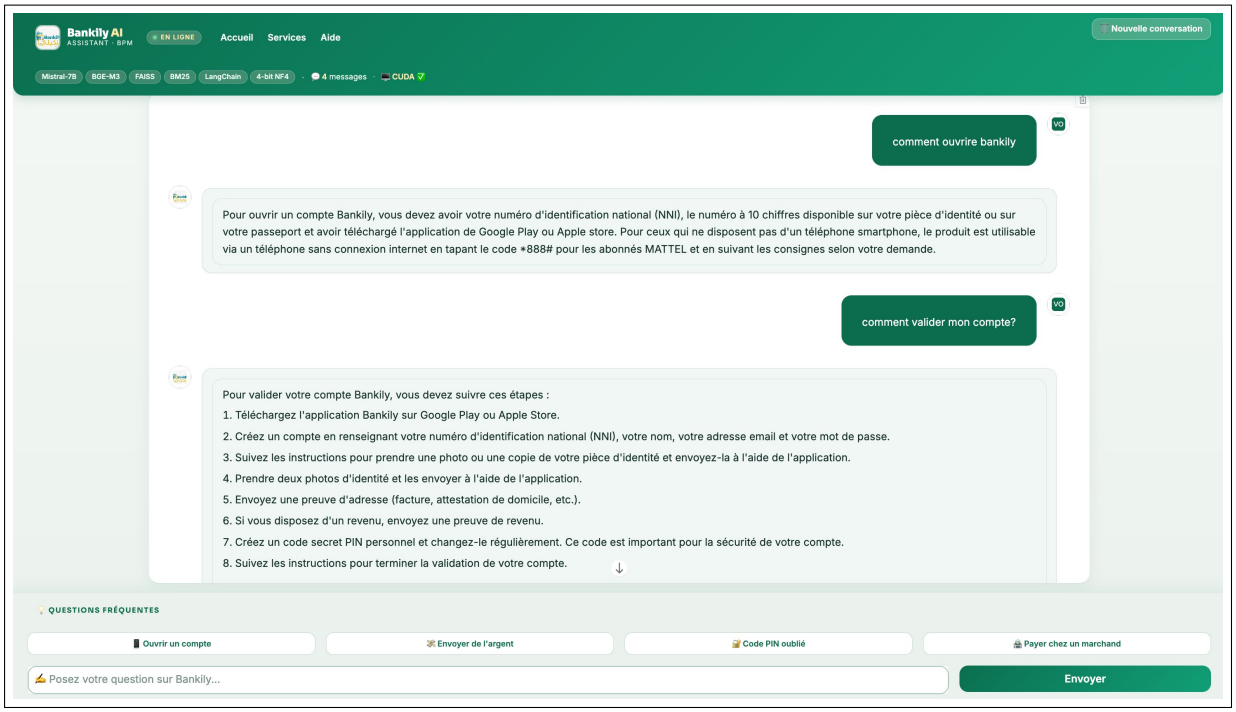


Figure 2: Interface conversationnelle de l'assistant Bankily AI développée avec Gradio. En-tête indiquant la pile technique (Mistral-7B, BGE-M3, FAISS, BM25, LangChain, quantification 4-bit NF4) et le périphérique GPU (CUDA). La session montre une question d'ouverture de compte puis une question de suivi sur la validation, traitées avec mémoire conversationnelle.

5.4 Fonctions principales

Table 4: Principales fonctions du projet et leur rôle.

Fonction	Rôle
`load_docs`	lecture et structuration des fichiers Markdown
`hybrid_search`	récupération hybride FAISS + BM25
`get_reranked_documents`	re-ranking par cross-encoder
`ask_bankily_with_memory`	génération de réponse contextuelle
`precision_recall_at_k`	métriques de récupération
`rag_answer`	branchement du pipeline sur l'interface Gradio

6 Méthodologie d'évaluation

6.1 Jeu de questions

Un jeu de **20 questions** a été constitué manuellement, couvrant l'ensemble des grands thèmes de la base (ouverture de compte, frais, transferts, paiements, services spécifiques). Chaque entrée associe à la question une réponse de référence (`ground_truth`) et la liste des fichiers attendus (`relevant_docs`). Ce jeu respecte le minimum de 20 questions exigé par le sujet. L'évaluation distingue volontairement deux dimensions : la qualité de la **récupération** et la qualité de la **génération**.

6.2 Métriques de récupération

Precision@3 proportion des 3 fragments re-classés qui proviennent du/des document(s) attendu(s) ;

Recall@3 proportion des documents attendus retrouvés parmi les 3 fragments.

6.3 Métriques de génération

Fidélité (BERTScore-F1) similarité sémantique entre la réponse générée et la réponse de référence, calculée par BERTScore (langue : français) ;

ROUGE-L recouvrement de la plus longue sous-séquence commune entre réponse et référence ;

Temps de réponse durée totale (recherche hybride + re-ranking + appel au LLM).

6.4 Reproductibilité

Pour reproduire l'évaluation : charger la base `.md`, construire les index FAISS et BM25, exécuter la boucle de récupération (`precision_recall_at_k`) sur les 20 questions, puis la boucle de génération qui mesure le temps de réponse et calcule BERTScore et ROUGE avec la bibliothèque `evaluate`. Les résultats présentés proviennent d'une exécution complète sur l'environnement de développement (Colab GPU, juin 2026).

7 Résultats et visualisations

L'évaluation a été menée sur les 20 questions du jeu de test. Le tableau 5 synthétise les moyennes obtenues, et la figure 3 en donne une vue d'ensemble.

7.1 Performance de la récupération

La figure 4 reproduit le nuage de points Precision@3 / Recall@3 (avec *jitter* pour éviter la superposition des points). Le recall moyen (0.68) est nettement supérieur à la precision moyenne (0.32) : le système retrouve souvent un fragment du bon document, mais deux des trois fragments

Table 5: Moyennes des métriques d'évaluation (20 questions).

Métrique	Valeur moyenne
Precision@3	0.32
Recall@3	0.68
Fidélité (BERTScore-F1)	0.71
ROUGE-L	0.16
Temps de réponse (s)	17.56

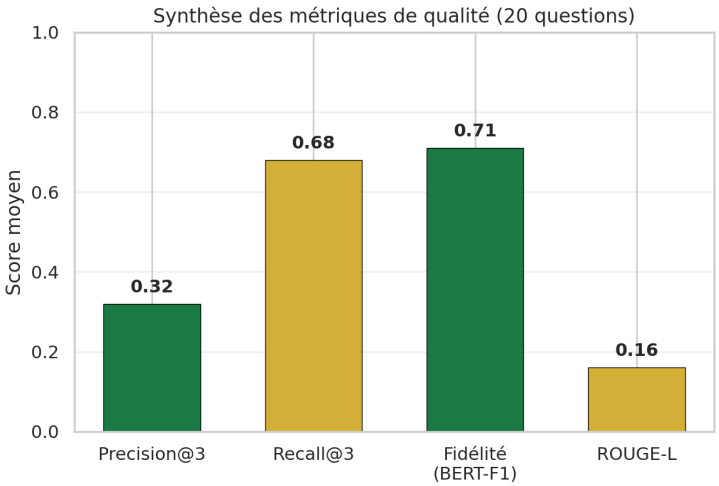


Figure 3: Synthèse des quatre métriques de qualité (récupération et génération).

retenus proviennent fréquemment de fiches voisines (différentes grilles tarifaires, FAQ et fiche d'ouverture de compte). La figure 5 détaille ces deux scores question par question : la plupart des questions atteignent un recall de 1.0 tandis que la precision reste souvent autour de 1/3, ce qui traduit l'effet de l'attribution d'un unique fichier « gold » par question.

7.2 Qualité et temps de génération

La figure 6 présente, comme dans le notebook, deux visualisations côte à côte : à gauche, la distribution des temps de réponse (histogramme avec courbe de densité) ; à droite, la distribution de la fidélité sémantique (BERTScore-F1) sous forme de boîte à moustaches.

La distribution des temps confirme une moyenne de **17,56 s**, dominée par l'inférence du modèle 7B quantifié en 4-bit sur GPU partagé, avec quelques réponses dépassant 30 secondes. La fidélité se concentre autour de **0,71**, indiquant que les réponses restent proches du sens des références. À titre de comparaison, le ROUGE-L moyen reste bas (0,16) : Mistral reformule largement et produit des réponses plus longues que les références courtes, ce qui réduit le recouvrement lexical sans nuire à la justesse sémantique.

> **Lecture des résultats.** Le système privilégie le *rappel* (retrouver l'information utile) et la *fidélité sémantique* (rester fidèle au sens des sources). La precision@3 et le ROUGE-L, plus bas, sont surtout pénalisés par des choix de protocole (un seul document attendu par question, métrique lexicale) plutôt que par une faiblesse réelle des réponses.

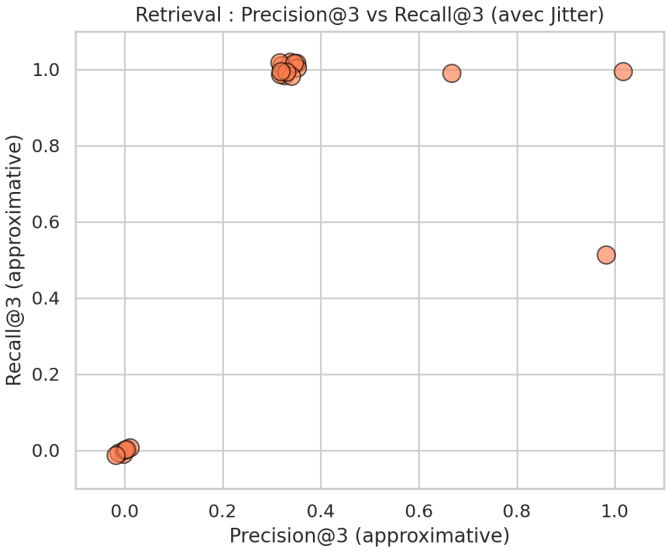


Figure 4: Nuage de points Precision@3 vs Recall@3 sur les 20 questions (avec jitter).

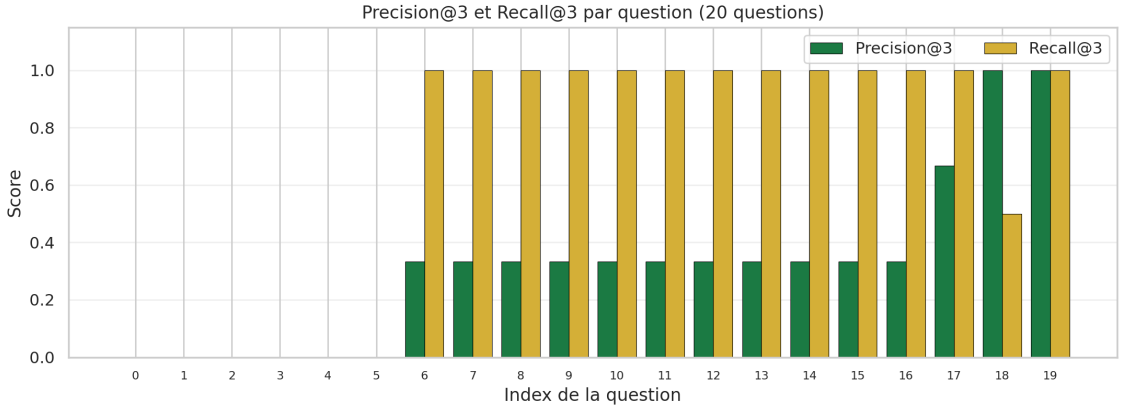


Figure 5: Precision@3 et Recall@3 pour chacune des 20 questions.

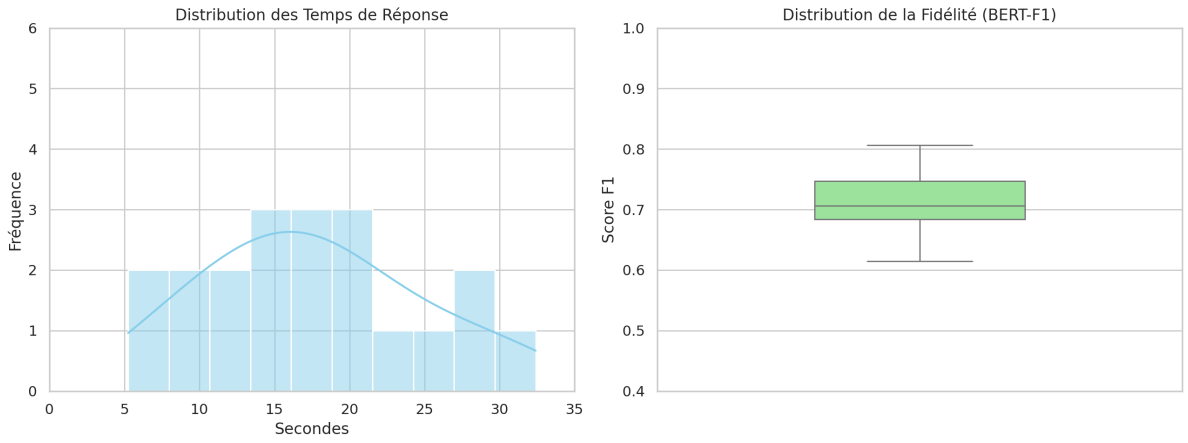


Figure 6: À gauche : distribution des temps de réponse (secondes). À droite : distribution de la fidélité BERTScore-F1.

8 Apport du RAG : comparaison avant / après

Pour démontrer concrètement la valeur ajoutée de l'approche RAG, le même modèle Mistral-7B-Instruct a été interrogé sur des questions identiques dans deux conditions :

- **Sans RAG** : le modèle répond uniquement à partir de ses connaissances internes (paramètres appris pendant son pré-entraînement), sans aucun document fourni ;
- **Avec RAG** : le modèle reçoit en contexte les fragments récupérés dans la base de connaissances Bankily avant de répondre.

Cette comparaison met en évidence le rôle de la récupération documentaire pour ancrer les réponses dans des informations exactes et propres au contexte mauritanien.

8.1 Question 1 : « C'est quoi Bankily ? »

Sans RAG (connaissances internes du modèle).

Réponse de Mistral-7B seul : « Bankily serait une entreprise **camerounaise** de paiement et de banque en ligne, permettant de gérer ses finances, d'effectuer des transferts et de payer des factures depuis un smartphone... »

→ **Réponse incorrecte et vague.** Le modèle hallucine une origine (« camerounaise »), reste générique et ne mentionne ni la Banque Populaire de Mauritanie (BPM), ni les spécificités réelles du service.

Avec RAG (contexte Bankily récupéré).

Réponse de l'assistant : « Bankily est le produit de Mobile Banking de la Banque Populaire de Mauritanie (BPM). Il vous permet de réaliser un ensemble d'opérations bancaires. »

→ **Réponse correcte et ancrée** dans la documentation officielle : bonne institution (BPM), bon pays, définition fidèle au service.

8.2 Question 2 : « Comment ouvrir / valider un compte Bankily ? »

Sans RAG.

Réponse de Mistral-7B seul : le modèle décrit une procédure générique et inventée (« aller sur le site `www.bankily.com`, cliquer sur *Sign Up*, alimenter le compte par carte bancaire... »), rédigée en **anglais** et sans rapport avec le fonctionnement réel.

→ **Réponse hallucinée :** site web inexistant, étapes erronées, mauvaise langue ; aucune mention du NNI ni des codes USSD.

Avec RAG.

Réponse de l'assistant : « Pour ouvrir un compte Bankily, vous devez disposer de votre numéro d'identification national (NNI, 10 chiffres) et avoir téléchargé l'application sur Google Play ou l'App Store. Pour les utilisateurs sans smartphone, le service est accessible via un téléphone classique en composant un code USSD... » suivie des étapes de validation (création du compte avec le NNI, envoi de la pièce d'identité, preuve d'adresse, création d'un code PIN personnel).

→ **Réponse correcte et complète**, fidèle à la procédure réelle de Bankily.

8.3 Synthèse de la comparaison

Le tableau 6 résume les différences observées.

Table 6: Comparaison des réponses du modèle sans et avec RAG.

Critère	Sans RAG	Avec RAG
Exactitude factuelle	Faible (origine et site inventés)	Élevée (BPM, NNI, USSD)
Spécificité au contexte	Générique, hors Mauritanie	Propre à Bankily / Mauritanie
Langue de réponse	Parfois incorrecte (anglais)	Conforme à la question (français)
Risque d'hallucination	Élevé	Limité (ancrage documentaire)
Vérifiabilité (sources)	Aucune	Sources affichées

Conclusion de l'expérience. À modèle identique, l'ajout de la récupération documentaire transforme des réponses génériques et parfois fausses en réponses exactes, spécifiques au service Bankily et vérifiables. C'est précisément l'objectif d'un système RAG : le modèle n'« apprend » pas par cœur, mais s'appuie sur la base de connaissances fournie pour répondre correctement.

9 Discussion

9.1 Points forts

- Le pipeline est fonctionnel de bout en bout : base de connaissances, double index, recherche hybride, re-ranking, génération et interface.
- La récupération hybride combinée au re-ranking offre un recall@3 correct (0.68) malgré la proximité sémantique de nombreuses fiches Bankily.
- La fidélité sémantique (BERTScore-F1 = 0.71) confirme que les réponses restent fidèles au sens des sources, et le prompt restrictif limite les hallucinations.
- Le système est multilingue (français/arabe) et conserve un historique de conversation, ce qui correspond au profil réel des utilisateurs de Bankily en Mauritanie.

9.2 Limites et travail en cours

- **Precision@3 faible (0.32)** : l'attribution d'un seul document « gold » par question pénalise mécaniquement la métrique lorsque des fragments pertinents proviennent de fiches voisines. Une annotation multi-documents serait plus juste.
- **ROUGE-L peu informatif** : métrique lexicale mal adaptée aux reformulations ; BERTScore reste plus pertinent ici.
- **Latence** : le temps de réponse moyen (17,56 s) est élevé, lié au LLM 7B quantifié.
- **Index volatil** : l'index FAISS est reconstruit à chaque session ; sa sauvegarde sur disque est prévue.

9.3 Alignement avec le sujet

Le projet satisfait les exigences principales : base de connaissances ≥ 30 documents, indexation vectorielle, pipeline RAG avec LLM, interface utilisateur, jeu de test ≥ 20 questions, et métriques de performance accompagnées de graphiques. L'ajout d'une récupération hybride et d'un re-ranking va au-delà du cadre minimal demandé.

10 Conclusion et perspectives

Ce projet a permis de mettre en œuvre l'ensemble de la chaîne d'un système RAG appliqué à un service mauritanien de monnaie électronique : constitution d'une base de connaissances Bankily, préparation des données, double indexation FAISS/BM25, récupération hybride suivie d'un re-ranking, génération contextuelle par Mistral-7B et interface conversationnelle Gradio.

Les résultats (Precision@3 = 0.32, Recall@3 = 0.68, BERTScore-F1 = 0.71, ROUGE-L = 0.16, temps moyen = 17,56 s) montrent un système fonctionnel produisant des réponses sémantiquement fidèles, dont les principaux axes d'amélioration concernent la précision de la récupération et la latence.

10.1 Perspectives

- Annoter les questions avec plusieurs documents pertinents pour une mesure plus juste de la precision.
- Persister l'index FAISS et mettre en cache les modèles pour réduire les temps de chargement.
- Réduire la latence (modèle plus léger, *batching*, ou API d'inférence dédiée).
- Étendre la base de connaissances et ajouter la citation explicite des sources dans la réponse.

11 Références

- Site officiel de Bankily — documentation et FAQ (consultés en juin 2026).
- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
- Jiang, A. et al. (2023). *Mistral 7B*.
- Chen, J. et al. (2024). *BGE-M3: Multilingual Embeddings*.
- Zhang, T. et al. (2020). *BERTScore: Evaluating Text Generation with BERT*. ICLR.
- Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*.
- Documentation FAISS — `https://faiss.ai/`
- Documentation LangChain — `https://python.langchain.com/`
- Documentation Gradio — `https://www.gradio.app/`

A Extraits de code

A.1 Prompt structuré (génération)

```
prompt_complet = f"""[INST] Tu es l'assistant expert de Bankily.
Consignes :
- Si tu ne connais pas la reponse a partir des documents, dis simplement
  que tu ne sais pas, n'invente pas de reponse.
- Reponds obligatoirement dans la meme langue que la question posee.

Historique de la conversation :
{history_text}

Documents de reference :
{context_text}

Question actuelle : {query} [/INST]"""
```

A.2 Recherche hybride (FAISS + BM25)

```python
def hybrid_search(query, k=10):
    # 1. Recherche semantique (FAISS)
    vector_results = vector_db.similarity_search(query, k=k)
    # 2. Recherche lexicale (BM25)
    tokenized_query = query.lower().split()
    bm25_scores = bm25.get_scores(tokenized_query)
    top_bm25_indices = np.argsort(bm25_scores)[::-1][:k]
    bm25_results = [documents[i] for i in top_bm25_indices]
    # 3. Fusion + suppression des doublons
    combined_results, seen = [], set()
    for doc in vector_results + bm25_results:
        if doc.page_content not in seen:
            combined_results.append(doc)
            seen.add(doc.page_content)
    return combined_results[:k]
```

A.3 Re-ranking par cross-encoder

```python
def get_reranked_documents(query, initial_docs, top_n=3):
    if not initial_docs:
        return []
    pairs = [[query, doc.page_content] for doc in initial_docs]
    scores = rerank_model.predict(pairs)
    for i, doc in enumerate(initial_docs):
        doc.metadata['rerank_score'] = scores[i]
    return sorted(initial_docs,
                  key=lambda x: x.metadata['rerank_score'],
                  reverse=True)[:top_n]
```

A.4 Métriques de récupération

```python
def precision_recall_at_k(retrieved_docs, relevant_docs, k=3):
    if not relevant_docs:
        return 0.0, 0.0
    retrieved_k = retrieved_docs[:k]
    sources = [d.metadata.get('source', '') for d in retrieved_k]
    hit = sum(1 for s in sources
              if any(rel.lower() in s.lower() for rel in relevant_docs))
    precision = hit / k
    recall = hit / len(relevant_docs)
    return precision, min(recall, 1.0)
```

B Exemples question-réponse

B.1 Exemple 1 — définition d'un service

Question : « C'est quoi Bankily ? »
Documents attendus : fiche de présentation.
Interprétation : les fragments récupérés proviennent des fiches de présentation et de la FAQ ; la réponse définit le service et son rôle dans l'inclusion financière.

B.2 Exemple 2 — question procédurale

Question : « Comment ouvrir un compte Bankily ? »
Documents attendus : `004_account_opening.md`.

Interprétation : la réponse restitue les étapes (application mobile, numéro valide, pièce d'identité), correctement ancrées dans le fragment d'ouverture de compte.

B.3 Exemple 3 — sources multiples

Question : « Comment effectuer un transfert international ? »
Documents attendus : `013_remitly.md`, `014_ria.md`.
Interprétation : cas où plusieurs fiches partenaires sont pertinentes ; le re-ranking favorise l'une d'elles, ce qui illustre l'écart entre precision et recall observé en section 7.